



Lecture 6 — Specifications

30 Exam-Style MCQs



Part 1: Purpose & Importance of Specifications

1.

What is the primary purpose of a software specification?

- A. To describe how the code is implemented
- B. To define what the software component is supposed to do
- C. To improve runtime performance
- D. To eliminate the need for testing

Answer: B

2.

Specifications are especially important because they:

- A. Reduce memory usage
- B. Act as a blueprint for expected behavior
- C. Automatically generate code
- D. Replace documentation

Answer: B

3.

Many serious software bugs arise mainly due to:

- A. Compiler limitations
- B. Poor performance optimization
- C. Misunderstandings at the interface between client and implementer
- D. Excessive testing

Answer: C

4.

A specification acts as a **contract** between:

- A. Compiler and runtime
- B. Tester and debugger
- C. Client and implementer
- D. User and operating system

Answer: C

5.

Why are specifications good for the *client* of a method?

- A. They allow modification of the implementation
- B. They eliminate the need to read the code
- C. They ensure faster execution
- D. They prevent runtime exceptions

Answer: B

6.

Why are specifications good for the *implementer*?

- A. They enforce a single implementation
- B. They restrict design choices
- C. They allow changing the implementation without affecting clients
- D. They eliminate the need for testing

Answer: C

7.

The “firewall” concept of specifications mainly results in:

- A. Tighter coupling
- B. Decoupling between client and implementation

- C. More runtime checks
- D. Reduced documentation

Answer: B

Part 2: Behavioral Equivalence

8.

Two methods are behaviorally equivalent if:

- A. They have identical code
- B. They always return the same value
- C. A client cannot distinguish between them under the specification
- D. They use the same algorithm

Answer: C

9.

Behavioral equivalence is defined from the perspective of the:

- A. Compiler
- B. Implementer
- C. Client
- D. Runtime system

Answer: C

10.

Why is a precise specification necessary for substituting implementations?

- A. To improve performance
- B. To guarantee memory safety
- C. To define exactly what behavior clients depend on
- D. To prevent runtime exceptions

Answer: C

11.

If a specification requires that `val` occurs exactly once in an array, then:

- A. Any implementation behavior is allowed
- B. The implementer must return the index of `val`
- C. The method may return arbitrary values
- D. The client must handle exceptions

Answer: B

Part 3: Specification Structure

12.

A method specification consists of:

- A. Code and comments
- B. Preconditions and postconditions
- C. Tests and assertions
- D. Parameters and return values only

Answer: B

13.

A **precondition** is an obligation on the:

- A. Compiler
- B. Implementer
- C. Client (caller)
- D. Runtime system

Answer: C

14.

A **postcondition** is an obligation on the:

- A. Client
- B. Implementer

- C. Tester
- D. Garbage collector

Answer: B

15.

If a precondition does **not** hold when a method is called, then:

- A. The compiler reports an error
- B. The method must throw an exception
- C. The implementation is not bound by the postcondition
- D. The program must terminate

Answer: C



Part 4: Specifications in Java

16.

Design by Contract (DbC) includes all EXCEPT:

- A. Preconditions
- B. Postconditions
- C. Invariants
- D. Garbage collection

Answer: D

17.

Java does not enforce Design by Contract automatically because:

- A. Java lacks exceptions
- B. Java relies on programmers to document and follow contracts
- C. Java only supports static checking
- D. DbC is deprecated

Answer: B

18.

In Java documentation comments, preconditions are usually described using:

- A. @return
- B. @throws
- C. @param
- D. @author

Answer: C

19.

A specification should never mention:

- A. Parameters
- B. Return values
- C. Local variables or private fields
- D. Exceptions

Answer: C

Part 5: Testing & Specifications

20.

Test cases must:

- A. Rely on the implementation
- B. Violate the precondition if needed
- C. Follow the specification
- D. Always test internal variables

Answer: C

21.

Why is this test incorrect?

```
assertEquals(0, find(array, 7));
```

- A. It is inefficient
- B. It assumes behavior not guaranteed by the spec
- C. It violates Java syntax
- D. It ignores return values

Answer: B

22.

Glass-box testing means:

- A. Ignoring the specification
- B. Testing only public methods
- C. Designing tests to exercise implementation paths while respecting the spec
- D. Using assertions instead of tests

Answer: C

23.

A good unit test should:

- A. Test multiple methods together
- B. Depend on other methods' correctness
- C. Focus on a single specification
- D. Replace integration tests

Answer: C



Part 6: Mutating Methods & Side Effects

24.

For mutating methods, the specification must describe:

- A. Local variables
- B. Side effects on mutable data

- C. Internal algorithms
- D. Execution time

Answer: B

25.

If mutation is not mentioned in the postcondition, it is assumed that:

- A. Mutation is allowed
- B. Mutation is optional
- C. Mutation is disallowed
- D. Mutation depends on implementation

Answer: C

Part 7: Designing Good Specifications

26.

A deterministic specification means:

- A. Multiple valid outputs per input
- B. Only one valid output for each valid input
- C. Undefined behavior
- D. Implementation-dependent behavior

Answer: B

27.

Underdetermined specifications are useful because they:

- A. Remove the need for testing
- B. Allow more implementation freedom
- C. Guarantee correctness
- D. Prevent runtime errors

Answer: B

28.

Declarative specifications are preferred because they:

- A. Explain how the method works internally
- B. Expose implementation details
- C. Describe what the result should satisfy
- D. Are longer and more detailed

Answer: C

29.

Specification S2 is **stronger** than S1 if:

- A. S2 has a stronger precondition
- B. S2 has a weaker postcondition
- C. S2 weakens the precondition and strengthens the postcondition
- D. Both have identical conditions

Answer: C

30.

Why should helper methods usually be private?

- A. To improve runtime performance
- B. To reduce memory usage
- C. To avoid exposing internal contracts and improve maintainability
- D. To allow external reuse

Answer: C